République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de L'enseignement Supérieur Et De La Recherche Scientifique
وزارة التعليم العالي والبحث العلمي

ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقا)
Ecole nationale Supérieure d'Informatique
Ex. INI (Institut National de formation en Informatique)

Multidisciplinary Project Report
2nd Year Preparatory Classes (2CP)
Project03 EQ39

Theme:

Heritage Community Algeria – Social Network for Heritage Enthusiasts

Prepared by:	
Last Name and First Name(s)	Groups
Team Leader	
SALAHOUELHADJ HALIMA	12
Members	
CHAKRANE FERIAL	12
MACHANE MAISSA	06
BENZITOUNI MEISSA	06
BENKHADRA OMAYMA	06

Teachers/Supervisors:
Mme.KHOURI Selma
Mr.Khouani Amin

Academic Year 2025 - 2026

Acknowledgements

First of all, we would like to express our deep gratitude to God, who guided and supported us throughout this semester, despite the unforeseen challenges. Without His grace and mercy, successfully completing this project would have been impossible.

We also warmly thank all the people who contributed, directly or indirectly, to the realization of this project. Our supervisors, Mrs. KHOURI and Mr. KHOUANI, deserve special recognition for their exceptional guidance, constant support, availability, and valuable advice. Their expertise and experience were a great source of inspiration and crucial for the success of this project.

Our parents also deserve our gratitude for their unconditional support and patience, which allowed us to overcome difficult moments and stay focused on our goal. Their love and encouragement were a driving force throughout this period.

Finally, our friends, relatives, and teachers at ESI deserve our sincere thanks for their support and enthusiasm at every stage of this project. Their confidence in us has been a constant source of inspiration, and we are proud to have them by our side.

We thank you all from the bottom of our hearts.

Table of Contents

Contents

1 Introduction **1**

2 Analysis of the Specifications **2**
2.1 User Management and Authentication 2
2.2 Publications, Content, and Social Interactions 2
2.3 News Feed, Search, and Civic Engagement 2
2.4 Administration, Interface, and Deliverables 2

3 Study of Existing Solutions **3**
3.1 Algerian and National Solutions 3
3.2 International Platforms 3
3.3 Use of General Social Networks for Heritage 4
3.4 Summary — Project Positioning 4

4 Study of the Environment **5**
4.1 User Environment 5
4.2 Technical Environment 5
4.3 Deployment Environment 5
4.4 Technical Constraints 5
4.5 Summary 6

5 Project Organization **7**
5.1 Work Methodology 7
5.2 Planning and Gantt Chart 7
5.2.1 Learning and Understanding the Specifications Phase 7
5.2.2 Design Phase 7
5.2.3 Development Phase 7
5.2.4 Application Finalization Phase 7
5.2.5 Writing Phase 8
5.3 Project Management Tools 8
5.3.1 Communication and Collaboration 9
5.4 Risk Management 11
5.4.1 Time Management 11
5.4.2 Front-End and Back-End Integration 11
5.4.3 Hardware Limitations 12

6 Conception **13**
6.1 Introduction 13
6.2 User Flow 13
6.2.1 Authentication and Access 13
6.2.2 Core User Experience 13

6.2.3 Community and Safety Features . . . 14
6.3 Database Schema . . . 15
6.4 Modular Breakdown . . . 15
6.4.1 Authentication Module . . . 16
6.4.2 Publication Module . . . 16
6.4.3 Social Interaction Module . . . 17
6.4.4 Search and News Feed Module . . . 18
6.4.5 Administration and Moderation Module . . . 19
6.4.6 System Outputs . . . 19
6.5 Graphic charter and visual identity . . . 20
6.5.1 Name and Logo . . . 20
6.5.2 Typography (Font) . . . 20
6.5.3 Color Palette . . . 20
6.6 UI UX Design . . . 21
6.6.1 Introduction . . . 21
6.6.2 Designing the User Interface . . . 22
6.6.3 User Experience Considerations . . . 22
6.6.4 Prototyping and Visualization . . . 22
6.6.5 Advantages of Using Figma . . . 22

7 Implementation **23**
7.1 Introduction . . . 23
7.2 Tools and Technologies Used . . . 23
7.3 Database . . . 24
7.3.1 Introduction . . . 24
7.3.2 Local Database Management . . . 24
7.3.3 Cloud Database Integration . . . 24
7.3.4 Database Structure . . . 24
7.3.5 Advantages of the Chosen Solution . . . 24
7.4 Front-end . . . 25
7.4.1 Introduction . . . 25
7.4.2 Frontend Implementation According to the Design . . . 26
7.4.3 Front-end Presentation . . . 27
7.5 Back-end . . . 29
7.5.1 Introduction . . . 29
7.5.2 Application Logic Using Dart and Dart Frog . . . 29
7.5.3 Code Structure and Architecture . . . 30
7.5.4 API Development (Endpoints) . . . 30
7.5.5 Database Integration . . . 30
7.5.6 Data Flow in the Application . . . 30
7.5.7 Authentication and Authorization . . . 31
7.5.8 Testing the Backend APIs . . . 31
7.5.9 Conclusion . . . 31
7.6 Integration . . . 32
7.6.1 Introduction . . . 32

7.6.2 Frontend and Backend Integration 32
7.6.3 Backend and Database Integration 32
7.6.4 Authentication Integration 32
7.6.5 Data Flow in the System 32
7.6.6 Conclusion 33
7.7 Deployment and Hosting 33

8 Conclusion **34**
8.1 Skills Acquired 34
8.2 Perspectives 34

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
UI	User Interface
UX	User Experience
REST	Representational State Transfer
RLS	Row Level Security
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
SQL	Structured Query Language
URL	Uniform Resource Locator
UML	Unified Modeling Language

1 Introduction

Members of the heritage community in Algeria—ranging from historians, students, and artists to passionate enthusiasts—devote significant effort to exploring cultural heritage, documenting traditions, and transmitting related knowledge. These activities require time for reflection and analysis in order to preserve the richness and diversity of Algerian heritage. Moreover, participation in cultural events and community initiatives plays an important role in fostering idea exchange and raising awareness about the value of this legacy.

Despite these efforts, the process of collecting, organizing, and sharing heritage-related content often remains inefficient. Many valuable ideas and observations are lost or remain confined to individual use, which limits opportunities for collaboration and collective benefit. This issue is largely due to the absence of a unified and interactive platform that facilitates the documentation and dissemination of such content among interested individuals.

In response to this challenge, the Heritage Community Algeria project proposes an innovative solution through the creation of a social network dedicated to heritage enthusiasts. The platform aims to provide an interactive digital space where users can document and share their ideas and experiences, as well as develop their knowledge of Algerian heritage, while fostering collaboration and exchange among different stakeholders.

In this report, we begin by analyzing the project specifications, focusing on its objectives and core functionalities. We will also present the adopted working methodology in terms of team organization and project planning, given their crucial role in ensuring the success of the project. Next, we will detail the application design, supported by mock-ups that illustrate an initial version of the system.

Finally, we will discuss the implementation phase, highlighting the technical choices made and presenting the contributions of both the frontend and backend teams, along with the results achieved and potential future developments.

2 Analysis of the Specifications

Following a thorough review of the project requirements, the goal of this application is clear: to design and develop a social web platform dedicated to Algerian architectural heritage. The system aims to centralize heritage-related information, facilitate its sharing among users, and foster an active community committed to the valorization and preservation of this cultural wealth.

2.1 User Management and Authentication

The platform will support full user account lifecycle management, enabling registration via email, secure login, and personalized profile pages that include a profile photo, biography, and area of expertise. The system will also handle password recovery and session management to ensure a secure and seamless experience.

2.2 Publications, Content, and Social Interactions

At the heart of the application lies a rich content publishing system. Users can create posts featuring descriptive text, up to five images, tags (covering historical period, monument type, and region), geolocation data, and detailed monument descriptions. Four content types are supported:

Discoveries, Visits, Questions, and Alerts about endangered monuments.

Users retain full control over their own posts through editing and deletion, and can report inappropriate content. Social interaction features — including likes, comments, sharing, and collaborative annotations — further enrich the community experience.

2.3 News Feed, Search, and Civic Engagement

The application will feature a chronological news feed with pagination and optional infinite scroll. Advanced filtering and search tools allow users to sort content by historical period, geographic location, heritage type, or keywords. A dedicated "Monuments in Danger" section encourages civic participation by letting users report endangered sites through a structured form that includes photos, urgency level, and status tracking.

2.4 Administration, Interface, and Deliverables

An administrator interface will provide moderation tools for managing reported content, user accounts (including suspension and banning), and platform statistics. The user interface will follow modern social network design principles — clean, responsive across all devices, and optimized for performance. At project completion, the expected deliverables include a full analysis and design report, user and installation guides, an integrated help system, the final web application, and supporting UML and architecture diagrams.

3 Study of Existing Solutions

The purpose of this study is to identify similar digital solutions already developed in the field of cultural heritage and thematic social networks, in order to understand the strengths and weaknesses of these platforms and better position our project.

3.1 Algerian and National Solutions

Portal of Algerian Cultural Heritage (patrimoineculturelalgerien.com)

Launched in February 2016 on the initiative of the Ministry of Culture, this multimedia portal aims to digitize cultural documents and make them accessible to the general public. It covers multiple areas including music, theater, books, poetry, cinema, and museums, and hosts a database containing over 10,000 audio files, 3,000 poetry texts, and more than 1,000 musical scores.

However, the portal suffers from fundamental limitations: it includes no social features whatsoever — no user accounts, no comments, no community interaction — updates are rare, and it does not cover architectural heritage at all.

Maps of Algerian Cultural Heritage (cartes.patrimoineculturelalgerien.org)

This initiative offers an interactive map of Algerian heritage sites such as Timgad, the M'zab Valley, and Tassili. While geographically valuable, it is strictly consultative, with no community participation or social dimension of any kind.

3.2 International Platforms

Historypin

A platform that allows individuals and communities to share their stories, memories, and photographs on an interactive map, enabling users to explore different locations and historical periods and uncover forgotten narratives behind familiar places. Its main strength is the geographic anchoring of content. Its limitations are significant, however: the interface is outdated, the mobile app has been removed from app stores, and the platform makes no accommodation for the Algerian or Arabic-speaking context.

Open Heritage (open-heritage.eu)

An independent online space open to contributions from researchers, practitioners, professionals, and citizens interested in promoting the value of cultural heritage. The platform provides multidisciplinary channels for exchanging expertise between individuals and institutions. That said, it lacks the features of a full social network — there is no news feed, no reactions, and no community profiles.

Google Arts ant Culture

An initiative that allows users to explore iconic heritage sites in 3D and discover digital preservation tools. Technically advanced and visually rich, it is nonetheless entirely editorial in nature: all content is produced by Google and partner institutions, with no user-generated contribution.

TripAdvisor

A general-purpose tourism platform supporting geolocation and user reviews. Although it covers some Algerian sites, it has no heritage specialization and lacks documentation tools

or any form of civic mobilization.

3.3 Use of General Social Networks for Heritage

In the absence of a dedicated platform, Algerians currently rely on mainstream social networks to share heritage content. On Instagram, location tags form visual databases of heritage sites, while Facebook groups enable collaborative documentation of local traditions. Diplomatic sources in the Maghreb have noted that these networks amplify demands that were once confined to specialists, turning them into political and mobilizing issues.

This dynamic confirms that strong community demand exists, but that general-purpose tools like Facebook and Instagram are inadequate for the purpose: they lack specialized heritage categorization, structured geolocation, thematic moderation, and any equivalent of an "endangered monuments" section.

3.4 Summary — Project Positioning

The study of existing solutions reveals a clear gap: no platform currently combines a community social network, specialization in Algerian architectural heritage, and civic mobilization tools. Our project distinguishes itself on three key axes.

The first is local specificity: content is centered on Algeria, with detailed categorization by region, historical period, and monument type. The second is a complete social dimension, encompassing user profiles, a news feed, reactions, comments, and sharing — all of which are entirely absent from existing institutional portals. The third is civic mobilization, through an "Endangered Monuments" section that is unique in the Algerian digital landscape.

4 Study of the Environment

4.1 User Environment

The project targets diverse categories of users that vary in their levels of expertise and requirements. These include heritage enthusiasts and students, researchers and academics specializing in history and architecture, architects and tourist guides, as well as ordinary citizens wishing to contribute to the documentation and preservation of cultural heritage. The majority of these users rely on modern web browsers across desktop computers, tablets, and smartphones, which requires the interface to be responsive and capable of adapting to various screen sizes.

4.2 Technical Environment

The technical architecture of the project is composed of three integrated layers.

Frontend Layer — Flutter

The team adopted the Flutter framework to build the frontend. Flutter enables the development of cross-platform applications from a single codebase, targeting both web browsers and mobile devices simultaneously. The code is written in Dart, and this layer is characterized by the richness of its visual components and rendering speed, ensuring a smooth and consistent user experience across all platforms.

Backend Layer — Dart Frog

The Dart Frog framework forms the backbone of the server side. This framework relies on a file-based routing system, which simplifies the creation and management of endpoints. The server handles incoming HTTP requests, applies business logic, and communicates with the database through Supabase.

Data Layer — Supabase

The platform uses Supabase as its comprehensive data management solution. Supabase is built on a PostgreSQL database and automatically provides a RESTful API for all tables, along with a built-in authentication and session management system, a storage service for files and images uploaded by users, and a fine-grained row-level permissions system known as Row Level Security.

4.3 Deployment Environment

In accordance with the project requirements set by ESI, the project must be delivered as a ready-to-run Docker container. This container encapsulates the Dart Frog server with all its dependencies, ensuring that the application can be installed and run on any machine without the need for a specific development environment setup and without any conflicts between different environments.

4.4 Technical Constraints

Images uploaded by users require compression and optimization to ensure fast loading times and reduce the load on the storage service. Additionally, Supabase enforces a Row Level

Security policy to ensure that each user can only access data they are authorized to view, thereby strengthening platform security and protecting user privacy.

4.5 Summary

The project embodies a three-layer architecture: Flutter on the frontend communicates with Dart Frog on the backend via HTTP requests, while Dart Frog in turn queries and manages data through Supabase. The backend is fully deployed inside a Docker container, resulting in a modular, scalable, and maintainable architecture.

5 Project Organization

5.1 Work Methodology

A progressive work methodology was adopted based on dividing the project into short phases and specific tasks, which made the organization of work and progress tracking easier.

At the beginning, the User Interface and User Experience (UI/UX) were designed to define the overall appearance of the application and its usage flow. After that, the work was divided into several tasks, where each task was assigned to one Front-end developer and one Back-end developer working in parallel.

The Front-end developer was responsible for creating the interface related to the task, while the Back-end developer handled the business logic, database operations, and API development for the same task. Once completed, both parts were integrated and tested before moving to the next task. This methodology helped speed up the development process and improve coordination between team members.

5.2 Planning and Gantt Chart

The project development was organized into several important phases to ensure proper progress and successful completion. Each phase focused on a specific objective during the realization of the application

5.2.1 Learning and Understanding the Specifications Phase

During this phase, the project requirements and objectives were analyzed carefully. Research was conducted to understand the users' needs, the technologies required, and the overall functionality of the application.

5.2.2 Design Phase

In this phase, the structure and visual appearance of the application were designed. Wireframes, interface layouts, color selection, and user experience considerations were prepared to create an intuitive and attractive design.

5.2.3 Development Phase

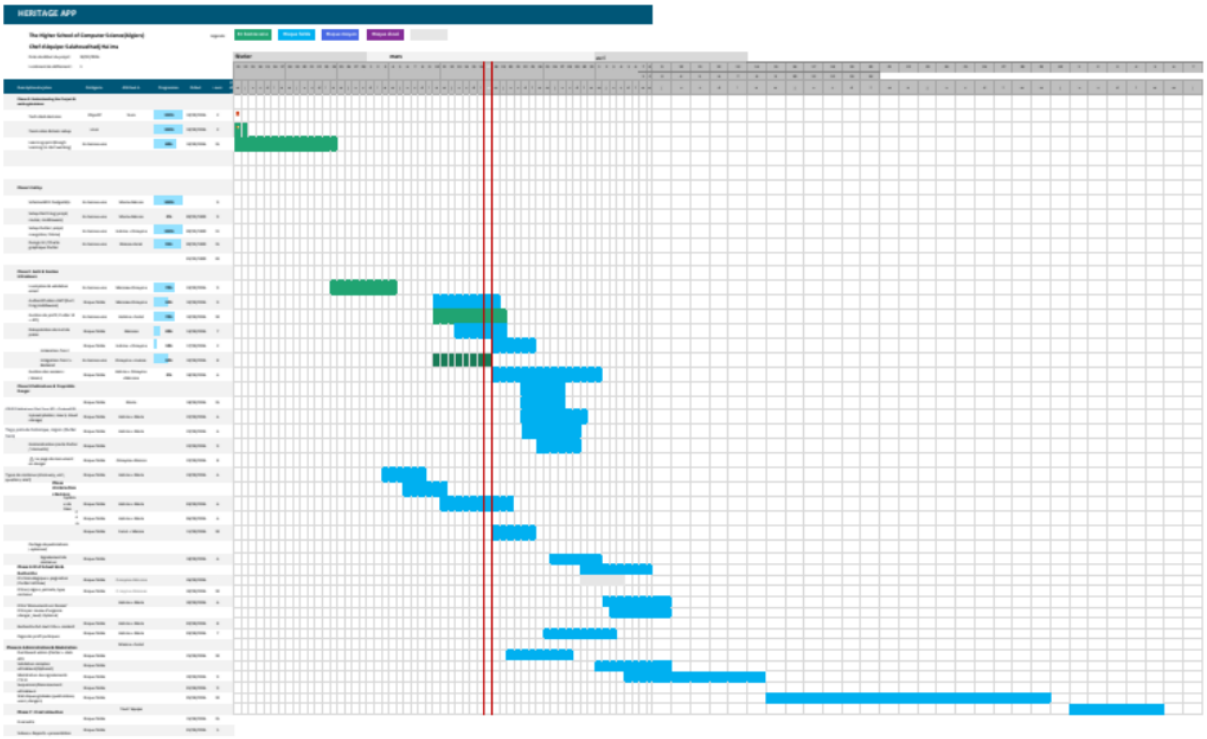
This phase focused on implementing the application features using the selected technologies. Both frontend and backend functionalities were developed, tested progressively, and integrated together.

5.2.4 Application Finalization Phase

During the finalization phase, the application was optimized and refined. Bugs were fixed, performance improvements were made, and final testing was completed before deployment.

5.2.5 Writing Phase

This phase involved preparing the project documentation and writing the final report, including explanations of the methodology, implementation process, results, and conclusions.

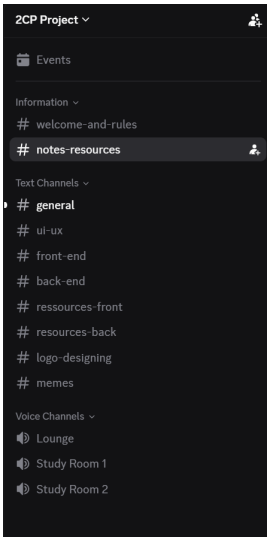


5.3 Project Management Tools

To enhance our organization and maximize efficiency, we implemented a comprehensive set of project management tools, which can be categorized into three main areas:

5.3.1 Communication and Collaboration

Discord is an online communication platform originally designed for gamers but now widely adopted by various communities and professional teams. It enables real-time text and voice communication, along with seamless file and media sharing.
We leveraged Discord by creating a dedicated server where all project-related information was centralized. This space served as a hub for sharing relevant resources, conducting discussions about ongoing tasks, and organizing meetings, work sessions, and collaborative exchanges. By utilizing structured channels and customized permission settings, we ensured clear, well-organized communication, allowing information to flow efficiently and reach the right team members at the right time.



Google Meet enabled seamless and efficient collaboration among team members during project meetings. It facilitated productive work sessions and ensured clear, transparent communication throughout the project. Its intuitive interface and smooth integration with other digital tools significantly improved coordination and enhanced overall team performance.

Gmail played a central role in facilitating formal communication within the project team. It was used to exchange important documents, share updates, and ensure reliable information flow between members. Its organization features, such as labels, filters, and threaded conversations, helped maintain clarity and structure in all project-related correspondence, improving efficiency and traceability.

Google Drive was used as a centralized storage system for all project files and resources. It allowed us to store, organize, and share documents, assets, and references securely. This ensured that every team member had access to the latest versions of files at any time, improving consistency and avoiding duplication or loss of data.

Google Docs was essential for collaborative documentation. It allowed multiple team members to write, edit, and comment on the project report simultaneously. This real-time collaboration improved efficiency and ensured consistency in the final documentation.

Microsoft Excel was used as an effective tool for project planning and task management through the creation of a Gantt chart. It allowed us to organize tasks clearly, define their start and end dates, and visualize the overall project timeline. This helped each team member understand their responsibilities and deadlines. The Gantt chart also enabled us to track the progress of tasks throughout the project and quickly identify any delays or overlaps. Additionally, Excel provided the flexibility to update the schedule when needed, ensuring that our planning remained accurate and up to date. Overall, it contributed to better time management, coordination, and organization within the team.

GitHub served as the central platform for version control and collaborative development throughout the project. It allowed us to manage the codebase in an organized way using branches, enabling each team member to work on separate features independently without interfering with others' work. Regular commits were used to document progress and track changes, which made it easy to review modifications and revert to previous versions when necessary. GitHub also helped us maintain a clean workflow and efficiently merge contributions from different members.

Figma played a crucial role in the design phase of the project. It enabled collaborative UI/UX design in real time, allowing the team to create, modify, and refine interface prototypes together. This helped us visualize the application structure early and ensure design consistency across all screens before development.

Canva was used to create visual content such as presentations and design elements. It helped us produce clean, attractive, and professional visuals, improving the overall presentation quality of the project.

Overleaf was the main platform used for writing and formatting the project report. It provided a collaborative LaTeX environment that allowed all team members to work on the same document simultaneously in real time. This made it easier to structure the report professionally, manage sections efficiently, and ensure consistency in formatting throughout the document. In addition, Overleaf simplified the process of handling references, equations, and layout organization, which are often complex in traditional word processors. Its cloud-based nature allowed us to access and edit the report from anywhere without the need for manual file sharing. Overall, Overleaf played a crucial role in producing a well-structured, clean, and professional final report.

Conclusion

Overall, the combination of these tools significantly enhanced our teamwork and productivity throughout the project. They allowed us to maintain clear communication, organize tasks efficiently, and collaborate seamlessly across different stages of development and design. By integrating platforms such as GitHub, Discord, Google Meet, and others, we were able to ensure a structured workflow, reduce misunderstandings, and improve coordination between all team members. This effective use of collaborative tools played a key role in the successful completion of our project.

5.4 Risk Management

Risk management is an essential component of any project, aimed at identifying, evaluating, and mitigating factors that may hinder the achievement of objectives. In this section, we detail the challenges encountered during the development of the Rawasii project and the strategies adopted to overcome them effectively.

5.4.1 Time Management

One of the primary difficulties faced at the outset of the project was time management. The team initially struggled to distribute tasks efficiently and maintain a consistent pace of development. However, this issue was progressively resolved through improved planning, clearer task allocation among team members, and the adoption of a more structured workflow that allowed the project to stay on track despite early setbacks.

5.4.2 Front-End and Back-End Integration

Another significant challenge arose during the integration phase between the front-end and back-end components of the application. Connecting the different parts of the system revealed compatibility issues and communication gaps between the two sides. These were addressed through better coordination between team members and more rigorous testing during the integration phase, ensuring the overall system functioned cohesively.

5.4.3 Hardware Limitations

On the technical side, one team member experienced performance issues with Flutter development due to insufficient RAM on her laptop, which disrupted her ability to run the application locally. This was resolved by turning to online compilation tools, enabling her to continue contributing to the project without interruption and without compromising the overall development timeline.

Overall, these challenges served as valuable learning experiences. They strengthened the team's problem-solving capabilities, fostered better communication, and reinforced collaboration — all of which contributed positively to the quality and resilience of the final product.

6 Conception

6.1 Introduction

The design phase represents the core foundation of any project and a decisive step toward its success. It plays a crucial role in clarifying project objectives, defining system functionalities, and evaluating different possible solutions to address the identified problem, all based on a thorough analysis of the specifications document. During this phase, key architectural and functional decisions are made to guide the development process effectively.

The primary goal of this stage is to ensure that the final product accurately satisfies user requirements, remains technically feasible within the available resources, and complies with time constraints. A well-executed design phase significantly reduces risks during implementation and improves the overall quality, coherence, and efficiency of the final system.

6.2 User Flow

In project design, the user flow plays a fundamental role in creating an optimal user experience and ensuring the overall success of the system. Also known as the user journey, it represents the sequence of steps and actions a user follows to achieve a specific goal within an application. It is typically illustrated as a diagram that maps the path taken by the user and highlights their interactions with the system at each stage of the process.

6.2.1 Authentication and Access

This is the first step when the user interacts with the application. The user can either sign up or log in. During sign up, the user provides basic information such as name, email, and password. After successful authentication, the user is redirected to the home page.

6.2.2 Core User Experience

This section represents the main functionalities of the application that users interact with daily.

- Home Feed

  Displays posts related to tourist landmarks in Algeria. Users can view shared content from other users.

- Popular Section

  Shows the most viewed and most engaging posts. Helps users discover trending content.

- Search

  Users can search for:

  Posts

  Tourist landmarks

  This allows quick access to specific content.

- Create Post

  Users can create posts about tourist landmarks.

  Posts may include text and images.

  Published posts appear in the main feed.

- Comments

6.2.3 Community and Safety Features

This section focuses on collaboration, awareness, and platform safety.

- Endangered Landmarks

  Displays landmarks that are at risk or endangered.

  Includes awareness information about their importance.

  Users can organize campaigns to support and protect these landmarks.

- Groups

  Users can create or join groups.

  Groups allow sharing posts and discussions about landmarks.

  Encourages community interaction.

- Report System

  Users can report inappropriate posts or content.

  Reports are sent to the administration for review.

- Administration

  The admin manages the platform content and user activity.

  Reviews reported posts and comments.

  Takes actions such as removal of content or warnings.

  Oversees campaigns in the endangered landmarks section.

  Ensures safety, quality, and proper use of the application.

6.3 Database Schema

6.4 Modular Breakdown

The modular breakdown of a software system is a design approach that consists of dividing the system into autonomous and interconnected modules, each responsible for a specific

functionality. This allows for better organization, easier maintenance, and a greater ability to evolve the software over time. To structure Rawasii in a modular way, we began by carefully analyzing the project requirements and defining all functionalities during the design phase. We then opted for a grouping approach, organizing features first into systems, then into broader super-systems.

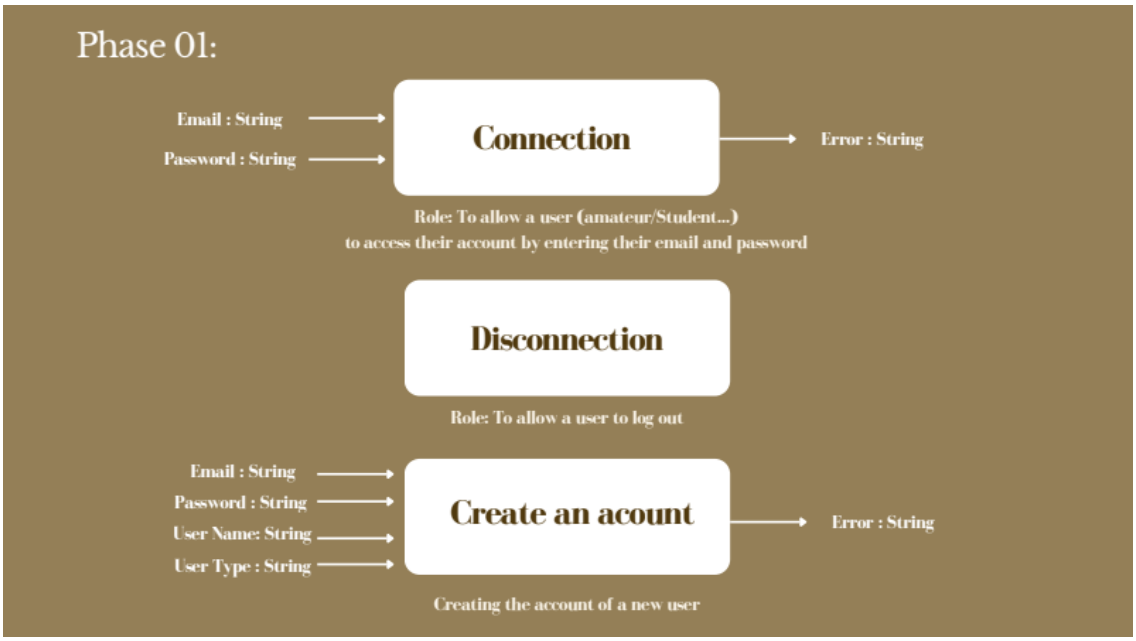
6.4.1 Authentication Module

The Authentication module represents the entry point of the Rawasii application, ensuring secure and controlled access to the platform. It is composed of three core functions:

The Connection function receives two inputs — the user's email and password — and grants access to their account upon successful validation. In case of invalid credentials, it returns an error message to inform the user. The Disconnection function allows any logged-in user to safely log out of the application, terminating their active session.

The Create an Account function handles the registration of new users by collecting four pieces of information: email, password, username, and user type. This last field distinguishes between different user roles on the platform. Like the connection function, it returns an error message if the registration process fails.

Together, these three functions form a complete and secure authentication flow that controls who can access Rawasii and in what capacity.



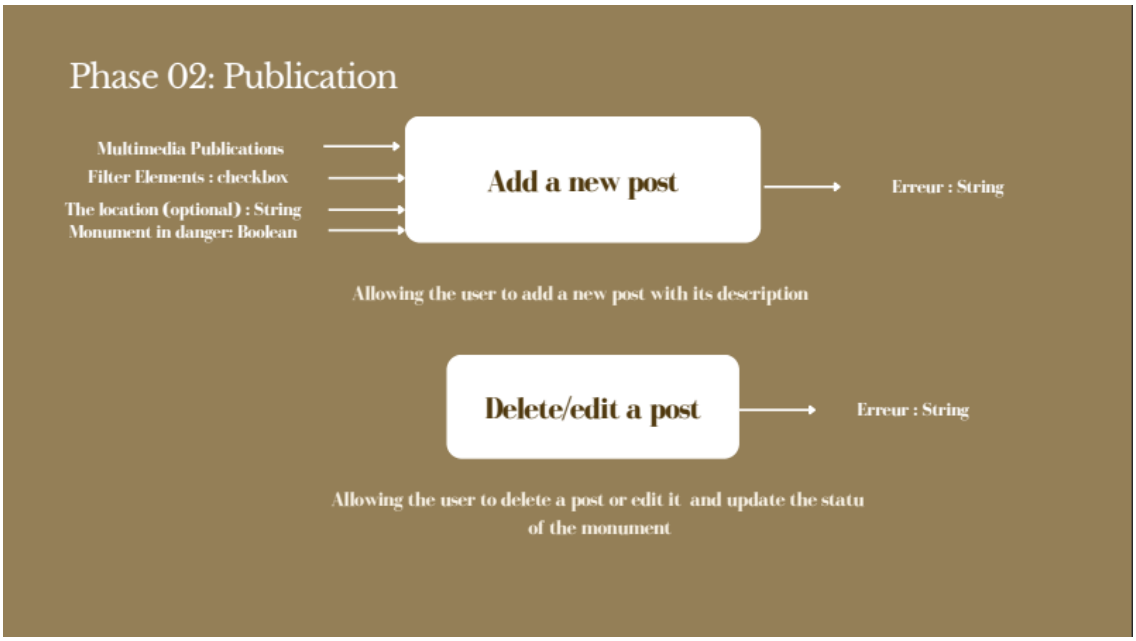
6.4.2 Publication Module

The Publication module enables users to share and manage heritage content on the platform. It consists of two main functions:

The Add a New Post function allows users to create and publish a heritage post by providing several inputs: multimedia content, filter elements through checkboxes for categorization,

an optional location, and a boolean field indicating whether the monument is in danger. This last field serves an important awareness purpose within the platform. The function returns an error message if the submission fails.

The Delete/Edit a Post function gives users control over their published content, allowing them to modify an existing post or remove it entirely, as well as update the status of the monument featured in the post. It also returns an error message in case of failure.

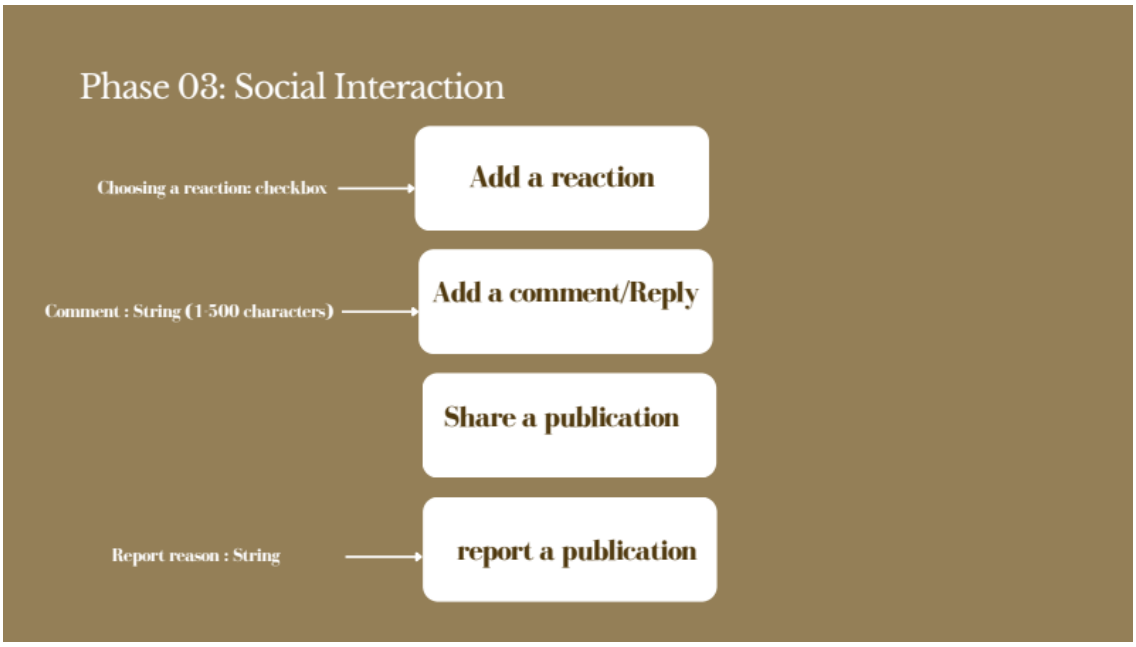


6.4.3 Social Interaction Module

The Social Interaction module covers all the ways users can engage with heritage content on the platform. It is composed of four functions: The Add a Reaction function allows users to express their feelings toward a post by selecting a reaction through a checkbox, enabling quick and expressive engagement.

The Add a Comment/Reply function lets users leave a comment or reply to an existing one, accepting a text input between 1 and 500 characters, ensuring meaningful yet concise interactions.

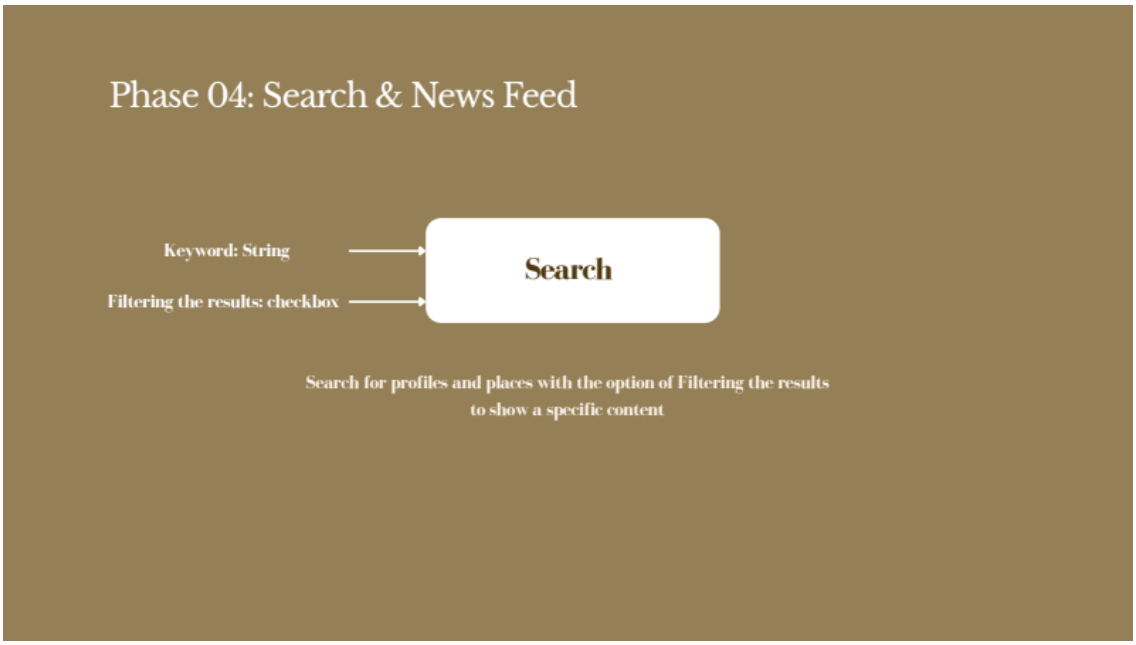
The Share a Publication function enables users to spread heritage content beyond the platform by sharing posts through various external channels. The Report a Publication function allows users to flag inappropriate or incorrect content by submitting a report reason as a text input, contributing to the quality and integrity of the platform.



6.4.4 Search and News Feed Module

The Search and News Feed module enables users to discover content and navigate the platform efficiently. It revolves around one core function:

The Search function accepts two inputs: a keyword as a text string, and filtering options through checkboxes to narrow down the results. This allows users to search for both profiles and heritage places, and to refine their results to display only the specific content they are looking for.



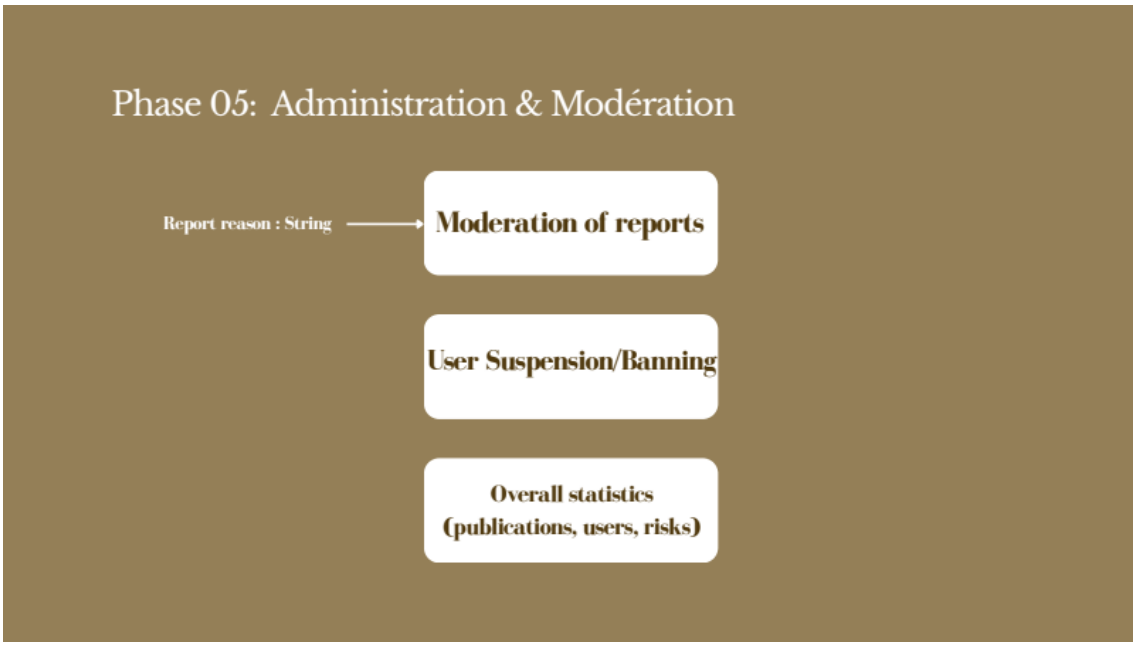
6.4.5 Administration and Moderation Module

The Administration and Moderation module grants platform administrators the tools needed to maintain a safe and trustworthy environment. It is composed of three functions:

The Moderation of Reports function allows admins to review user-submitted reports by examining the report reason provided as a text input, and taking appropriate action on flagged content.

The User Suspension/Banning function gives administrators the ability to restrict access for users who violate platform rules, either temporarily through suspension or permanently through banning.

The Overall Statistics function provides admins with a comprehensive overview of the platform's activity, covering key metrics related to publications, users, and at-risk monuments, enabling informed decision-making and platform monitoring.



6.4.6 System Outputs

The Rawasii system produces the following outputs:

Member Profiles — personalized user pages displaying identity, interests, and published content. Filtered News Feed — a paginated and filterable stream of heritage publications tailored to the user's preferences.

Heritage Map — an interactive map displaying geolocated monuments, with endangered sites clearly marked. At-Risk Publications — posts flagged by the danger indicator, filtered by urgency level to prioritize critical heritage alerts. Search Results — content filtered by historical period, region, type, and danger status to deliver precise and relevant results.

Notifications — real-time alerts covering replies, reactions, and annotation validations. Admin Dashboard — a centralized panel providing global statistics, reported content, and moderation tools for platform administrators.

6.5 Graphic charter and visual identity

6.5.1 Name and Logo

The name "Rawasii" was chosen because it reflects stability, authenticity, and deep roots, values that strongly relate to Algerian architectural heritage.

The word is inspired by the writings of Ibn Khaldun in Al-Muqaddimah, symbolizing firmness and cultural continuity across generations.

Our project aims to preserve and strengthen the connection between Algerians and their architectural heritage by presenting traditional monuments, designs, and historical identities in a modern digital form.

The logo was designed to visually represent Algerian architectural identity through elements inspired by traditional structures and cultural symbols, combining heritage with modern technology.

6.5.2 Typography (Font)

The Tajawal font was selected for this project because it is a modern typeface characterized by clarity, simplicity, and excellent readability across different screen sizes. It is also a versatile font that supports both Arabic and Latin scripts, making it suitable for an application intended for users from diverse regions around the world. This font contributes to enhancing the visual user experience by providing a clean and well-structured interface that reflects the modern identity of the application.

6.5.3 Color Palette

Colors play a fundamental role in this project, as each shade directly influences the user's emotions and perception. This has led to the selection of a coherent palette inspired by Algerian heritage, where each color conveys a specific meaning and contributes to the visual identity of the platform.

#2D1B15 : It helps organize the interface and reinforces information hierarchy.

#4A2C24 : It is used for titles and important elements to ensure strong readability.

#9C6B3F : This color reflects traditional elements such as pottery, wood, and desert landscapes.

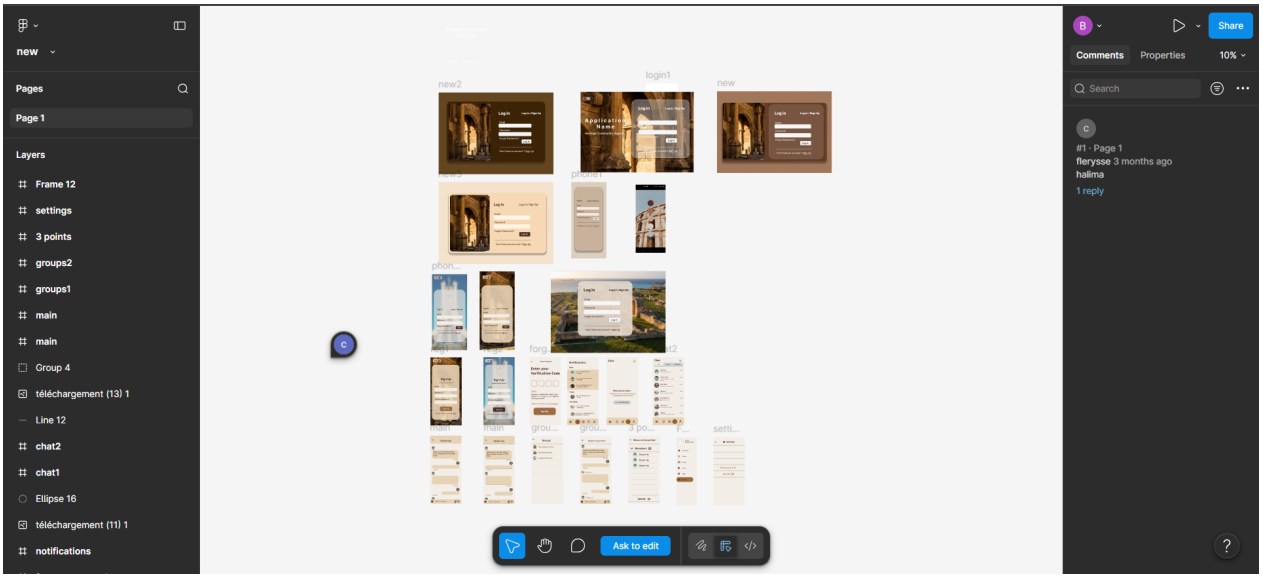
#C9B29B : It introduces a soft traditional touch inspired by natural and cultural Algerian tones.

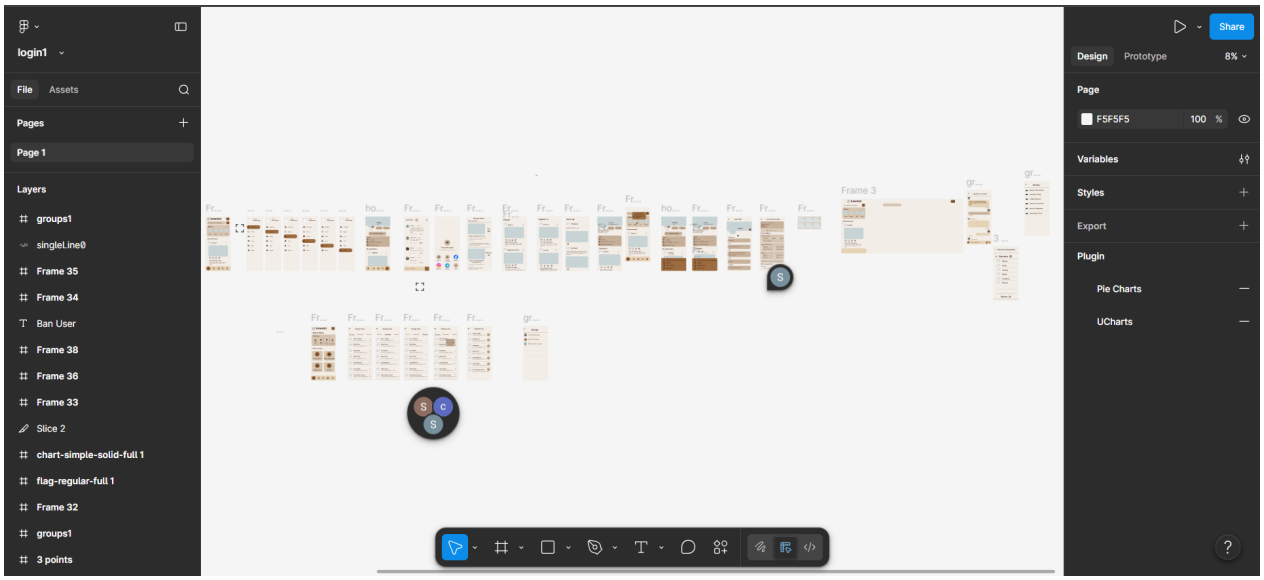
#F2EDE6 : It is used as the main background color to ensure comfortable reading and highlight content without distraction.

6.6 UI UX Design

6.6.1 Introduction

The UI/UX design phase focused on creating an interface that is visually attractive, easy to use, and provides a smooth user experience. Special attention was given to the organization of elements, navigation simplicity, and the overall appearance of the application in order to make the interaction intuitive and comfortable for users.





6.6.2 Designing the User Interface

The user interface was designed using Figma, which allowed the creation of organized and interactive layouts before starting the development phase. Through Figma, different screens of the application were designed while respecting consistency in colors, typography, spacing, and component placement.

The design process focused on creating a modern and clean interface that reflects the purpose of the application while maintaining simplicity and clarity.

6.6.3 User Experience Considerations

During the UI/UX phase, several user experience principles were taken into consideration to improve usability and accessibility. Navigation between screens was designed to be simple and intuitive, allowing users to interact with the application efficiently without confusion.

Attention was also given to the arrangement of buttons, menus, and content sections in order to provide a smooth and comfortable experience for different types of users.

6.6.4 Prototyping and Visualization

Figma was also used to create prototypes and visualize how users would interact with the application before implementation. This helped identify possible design improvements early in the project and reduced potential issues during development.

6.6.5 Advantages of Using Figma

Using Figma provided flexibility and efficiency during the design process. It simplified collaboration, accelerated interface creation, and allowed easy modification of components and layouts throughout the project development cycle.

7 Implementation

7.1 Introduction

Following a successful design phase, the project progresses to the implementation stage of the software. This phase involves developing the various functionalities independently on both the front-end and back-end levels, and then integrating them to produce a coherent and fully functional final product.

Special emphasis is placed on testing throughout this stage to refine, optimize, and enhance the system. This process ensures that the software meets quality standards, performs efficiently, and delivers a stable and reliable user experience.

7.2 Tools and Technologies Used

VS Code was used as the main development environment for writing and managing the project code. It provided a lightweight interface with useful extensions that improved productivity and simplified debugging.

Flutter was used as the frontend framework to build the user interface of the application. It enabled us to create a responsive and consistent UI across different devices using a single codebase.

Dart was the primary programming language used in the project. It was mainly used with Flutter and Dart Frog, ensuring efficient and structured development.

Dart Frog was used to develop the backend of the application. It allowed us to build APIs and handle server-side logic in a simple and organized way.

Git was used as a version control system to track changes and manage collaboration between team members. It helped maintain a clean and organized development workflow.

Supabase was used as the backend-as-a-service platform, providing a PostgreSQL database along with features such as authentication and real-time data handling, which simplified backend development and data management.

pgAdmin 4 was used as a database management tool to interact directly with the PostgreSQL database. It allowed us to create tables, manage data, execute SQL queries, and monitor the database structure in an efficient and user-friendly way.

Docker was used to containerize the application and ensure a consistent development and deployment environment. It allowed us to package the backend and its dependencies into isolated containers, making the application easier to run across different machines without configuration issues. Additionally, Docker simplified the setup process and improved portability, especially when working on different devices. It also helped maintain consistency between development environments and reduced potential compatibility problems.

7.3 Database

7.3.1 Introduction

The database is a fundamental component of the application, used to store and organize data in a way that ensures easy access, integrity, and efficient performance during usage.

7.3.2 Local Database Management

At the beginning of the project, pgAdmin 4 was used as a local PostgreSQL database management tool. It helped in creating tables, testing relationships between data, and executing various SQL commands during the early development phase, which provided a better understanding of the database structure before moving to the final environment.

7.3.3 Cloud Database Integration

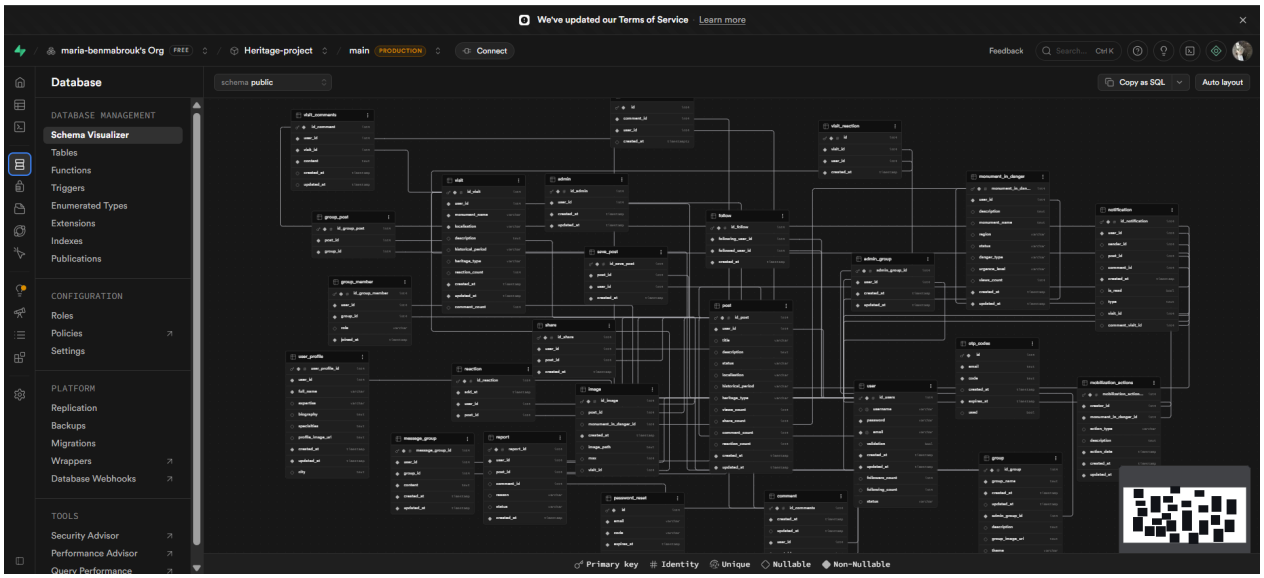
Later, the project was migrated to Supabase, a cloud-based platform built on PostgreSQL that provides an integrated environment for managing and connecting the database directly with the application. This transition allowed remote access to the database and improved development and deployment efficiency.

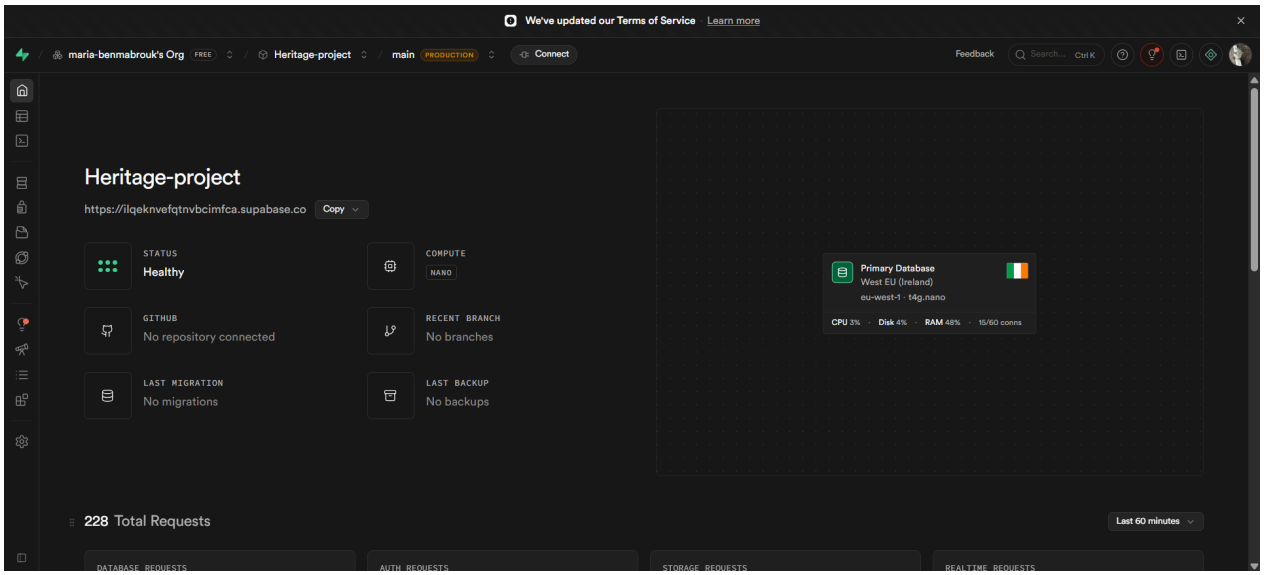
7.3.4 Database Structure

The data was organized into relational tables to reduce redundancy and ensure data integrity. Relationships between tables were designed logically to allow efficient and fast data management.

7.3.5 Advantages of the Chosen Solution

Using Supabase with PostgreSQL improved the application's performance by providing automatic APIs, authentication support, easier scalability, and a stable and secure data environment.





7.4 Front-end

7.4.1 Introduction

The front-end part of a web application represents the interface through which users directly interact with the system. It includes all visible and interactive elements of the application, as well as the functionalities associated with them. Front-end development plays an essential role in creating an intuitive, responsive, and engaging user experience.

In the development of the front-end part of our project, we chose to use Flutter and Dart. These modern technologies enabled us to build a dynamic and responsive application with an attractive and user-friendly interface.

Our project is a social networking platform dedicated to Algerian architectural heritage. The application allows users to share publications, interact with other members, participate in thematic discussions, and explore heritage-related content through a collaborative

environment.

For the realization of this important part of the project, we relied on Flutter's flexible UI system and Dart's programming capabilities to create modern interfaces and ensure smooth interaction between users and the application.

7.4.2 Frontend Implementation According to the Design

Translation of the Design into Flutter Widgets

We started by transforming the interface mockups into Flutter widgets. Each visual element was divided into reusable and modular components while paying particular attention to responsiveness and interface consistency.

Extensive Use of Flutter UI Components

Thanks to Flutter's rich collection of customizable widgets, we were able to easily translate the visual designs into code. By using modern UI components and organized styling practices, we successfully reproduced the desired interfaces while maintaining a clean and efficient code structure.

Integration of Navigation and User Interactions

We implemented navigation between the different sections of the application and integrated several interactive functionalities such as authentication, publications, likes, comments, notifications, group discussions, and search features. These interactions contributed to improving communication and engagement between users.

Continuous Testing and Optimization

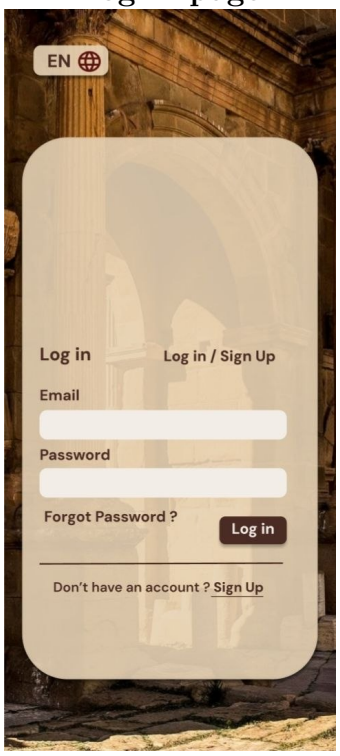
We conducted continuous testing to ensure the stability and performance of the application. This included testing the responsiveness of the interfaces, verifying user interactions, and optimizing the fluidity of navigation.

During development, we encountered some technical challenges related to Flutter implementation and integration between application components. However, these issues were progressively resolved through debugging and continuous improvements.

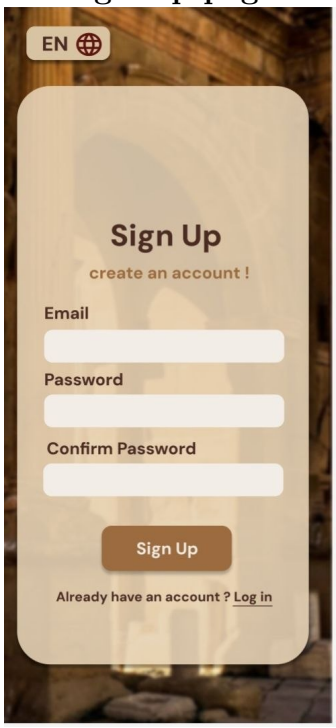
By following this approach, we successfully transformed our conceptual design into a functional and visually coherent front-end application. Every design decision was carefully implemented in the frontend development process, resulting in a modern and interactive user experience.

7.4.3 Front-end Presentation

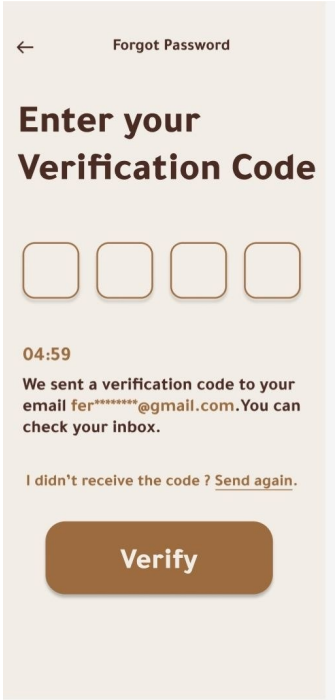
Log in page



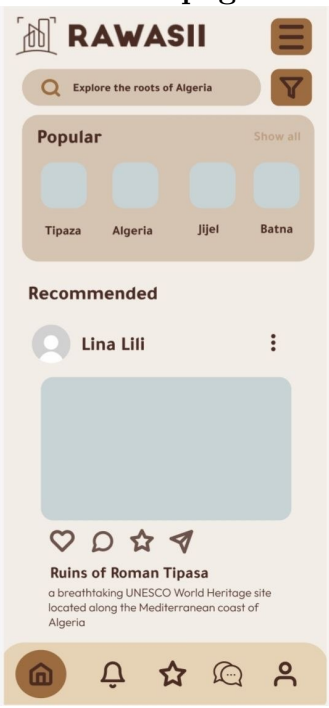
Sign Up page



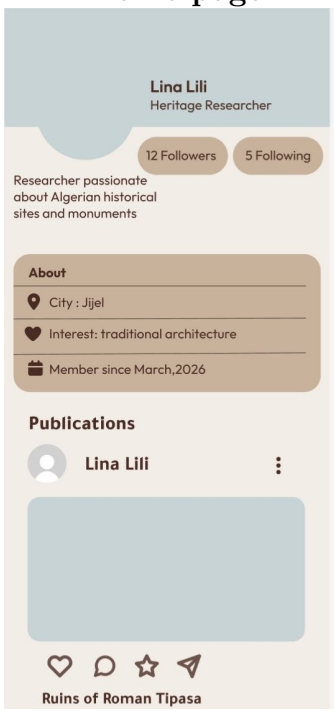
Forget password page



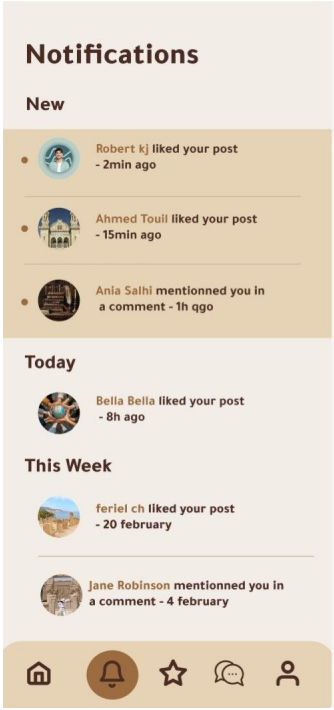
Home page



Profile page



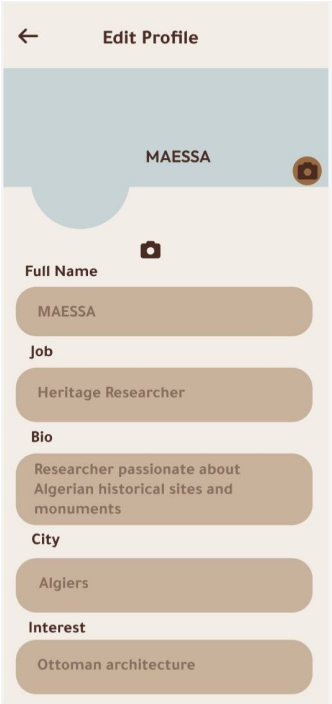
Notifications page



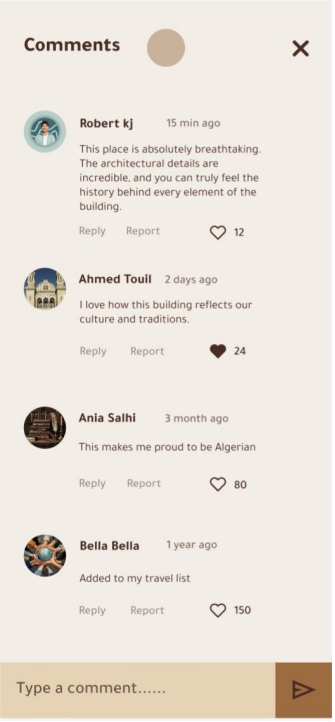
Admin panel page

Add post

Edit profile

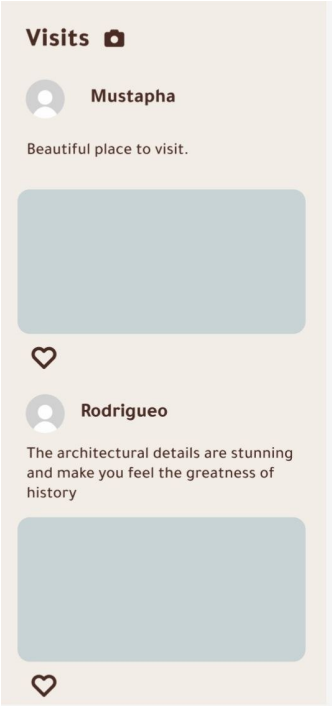


Comments

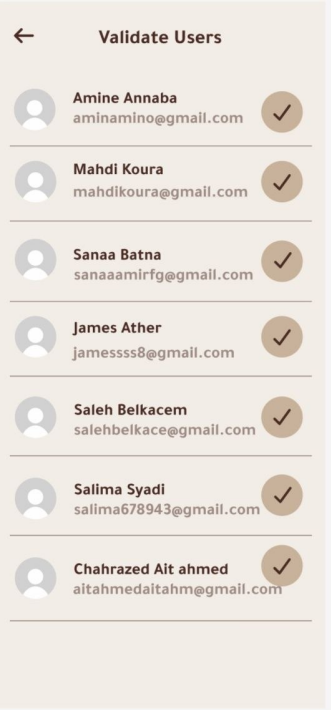


Statistics

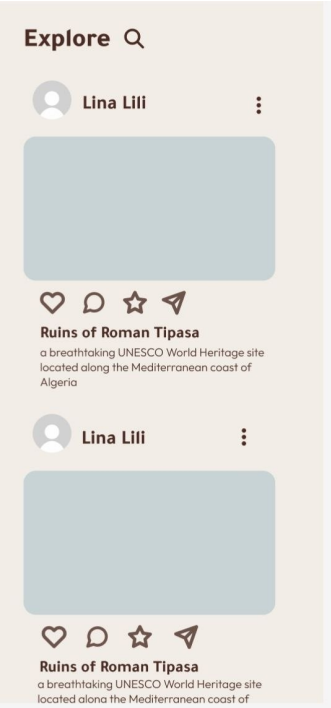
visits page



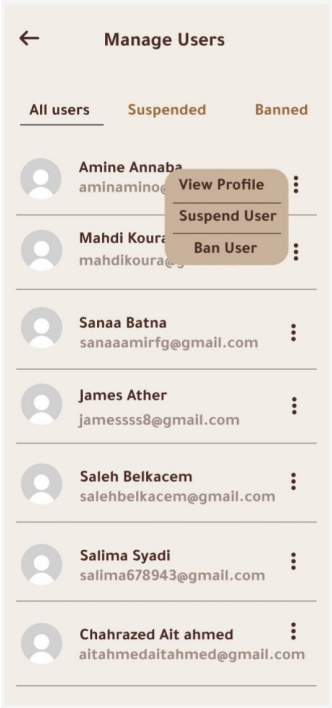
Validate users



Explore



Manage users



7.5 Back-end

7.5.1 Introduction

The backend is a core component of the application responsible for handling business logic, managing data, and enabling communication between the frontend and the database. It acts as an intermediary layer that receives user requests, processes them, and returns appropriate responses in a structured and secure way.

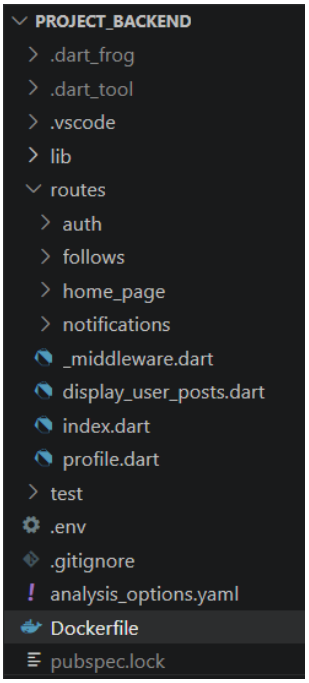
The backend was developed using the Dart programming language with the help of the Dart Frog framework, which provides a modern and lightweight environment for building RESTful APIs and server-side applications. In addition, Supabase was used to manage the database and authentication services in a cloud-based environment.

7.5.2 Application Logic Using Dart and Dart Frog

The Dart language was used to implement the application's core logic, including data processing, input validation, and controlling the flow of operations within the system. This allowed the creation of a clean and well-organized codebase that is easy to maintain and modify.

In addition, Dart Frog was used to build the backend of the application. This framework provides a modern way to create APIs using Dart, allowing the development of lightweight and fast REST-based servers. It also helped structure the project by separating responsibilities, making the code more readable and easier to maintain.

7.5.3 Code Structure and Architecture



The code was organized in a modular and structured way to improve readability, maintainability, and scalability. The project was divided into separate files and layers, where each part is responsible for a specific functionality such as request handling, database communication, and business logic.
This separation of concerns helped reduce code complexity and made it easier to maintain and extend the application in the future. Each module is independent, which allows developers to modify or add features without affecting other parts of the system.

7.5.4 API Development (Endpoints)

The backend APIs were developed using Dart Frog to handle communication between the application and the backend system. These APIs act as endpoints that receive HTTP requests from the frontend, process the data, and return appropriate responses.

A set of RESTful endpoints was created to support the main operations of the application, including retrieving data, adding new records, updating existing data, and deleting records.

Although Supabase provides built-in auto-generated APIs for database operations, custom endpoints were also implemented using Dart Frog to handle application-specific business logic and ensure better control over data processing.

7.5.5 Database Integration

Supabase was used as a cloud-based backend solution for database management. The application is directly connected to the Supabase database, enabling real-time operations such as inserting, retrieving, updating, and deleting data.

Supabase is built on PostgreSQL and provides an integrated environment that simplifies backend development by offering authentication, database management, and API services in one platform. The data is organized into relational tables to ensure consistency, reduce redundancy, and improve performance.

7.5.6 Data Flow in the Application

Requests are sent from the frontend to the Dart Frog backend, where they are received, processed, and validated. The backend then communicates with Supabase to retrieve or update data, and finally sends the response back to the application for display.

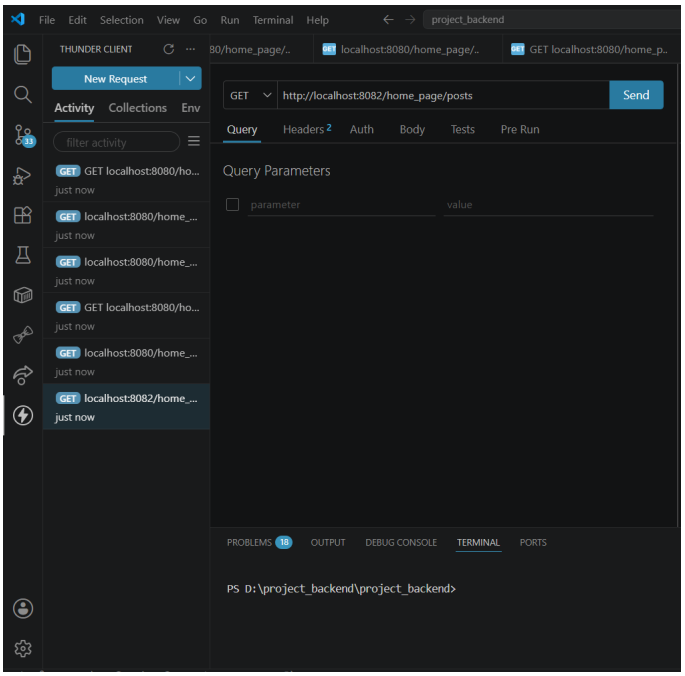
This flow ensures a well-structured system where data handling is organized, secure, and efficient.

7.5.7 Authentication and Authorization

User authentication was implemented using Supabase authentication services to securely manage user login and account creation. This ensures that only verified users can access the application. In addition to authentication, an authorization system was applied to control user permissions and define what actions each user is allowed to perform. This helps protect sensitive data and ensures proper access control within the application.

7.5.8 Testing the Backend APIs

To ensure the proper functioning of the backend, the APIs were tested using Thunder Client. This tool was used to send requests to different endpoints and verify that the backend built with Dart Frog is working correctly and is able to communicate with Supabase.
The testing process focused on checking the structure of the requests and ensuring that the endpoints respond as expected. Although only the request configuration is documented in the report, the tests confirmed that the API operations (such as data retrieval and submission) were successfully executed during development.



7.5.9 Conclusion

The backend system was successfully developed using Dart and the Dart Frog framework, combined with Supabase for database and authentication management. This combination resulted in a fast, scalable, and well-structured backend architecture.

The clear code organization, well-designed APIs, and secure authentication system contributed to building a reliable and efficient application capable of handling data operations effectively while ensuring maintainability and future scalability.

```dart
routes > display_user_posts.dart > onRequest
import 'package:dart_frog/dart_frog.dart';
import 'package:project_backend/supabase_client.dart';

Run | Debug
Future<Response> onRequest(RequestContext context) async{
  final method=context.request.method; //http request

  if(method!=HttpMethod.get){
    return Response.json(
      statusCode: 405,
      body: {'error': 'Method not allowed'},
    );

  }
  final idUser = context.request.uri.queryParameters['id_users'] ;
  if(idUser == null){
    return Response.json(
      statusCode: 400,
      body: {'error': 'User ID is required'},
    );
  }

  //pagination(page number)
  final pagePosts = int.tryParse(
  context.request.uri.queryParameters['page'] ?? '1',
  ) ?? 1;
  //number of posts per page
  const limitPagePosts =  2;
```

```dart
routes > home_page > comments visits > add_comment.dart > ...
Future<Response> onRequest(RequestContext context) async {
  final visit = await supabase
  .from('visit')
  .select('comment_count')
  .eq('id_visit', idVisit)
  .single();


  final currentCommentCount = visit['comment_count'] as int? ?? 0;


  await supabase
  .from('visit')
  .update({'comment_count': currentCommentCount + 1})
  .eq('id_visit', idVisit);


  if(idPostUser != idUser){
 await supabase
  .from('notification')
  .insert({
    'user_id': idPostUser,
    'type': 'comment_visit',
    'sender_id': idUser,
    'visit_id': idVisit,
    'comment_visit_id': addComment['id_comment'],
    'is_read':false,
    'comment_id':null,
```

7.6 Integration

7.6.1 Introduction

The integration phase focuses on connecting all parts of the application to ensure that the system works as a single unified product. This stage involves linking the frontend, backend, and database to enable smooth and accurate data exchange between all components.

7.6.2 Frontend and Backend Integration

The frontend was connected to the backend developed using Dart Frog through RESTful APIs. This integration allows the user interface to send requests to the server, where they are processed and appropriate responses are returned and displayed in the application.

This connection makes the application dynamic, as the displayed data changes based on user interactions in real time.

7.6.3 Backend and Database Integration

The backend was integrated with Supabase for cloud-based data management. Through this integration, core operations such as inserting, retrieving, updating, and deleting data were efficiently performed in an organized manner.

Supabase provided a ready-to-use database environment, which simplified communication between the backend and stored data while ensuring consistency and reliability.

7.6.4 Authentication Integration

The Supabase authentication system was integrated into the application to securely manage user registration and login. This integration ensures that only authenticated users can access specific features of the system, helping protect data and manage user permissions effectively.

7.6.5 Data Flow in the System

Data is exchanged through APIs built with Dart Frog. Requests are sent from the frontend to the backend, processed accordingly, and then the backend communicates with the database

when needed. Finally, the response is returned to the frontend to be displayed to the user.

7.6.6 Conclusion

The integration phase successfully connected all components of the application into a unified system. The use of Dart Frog and Supabase ensured smooth communication between layers, improved data organization, and provided a stable and efficient user experience.

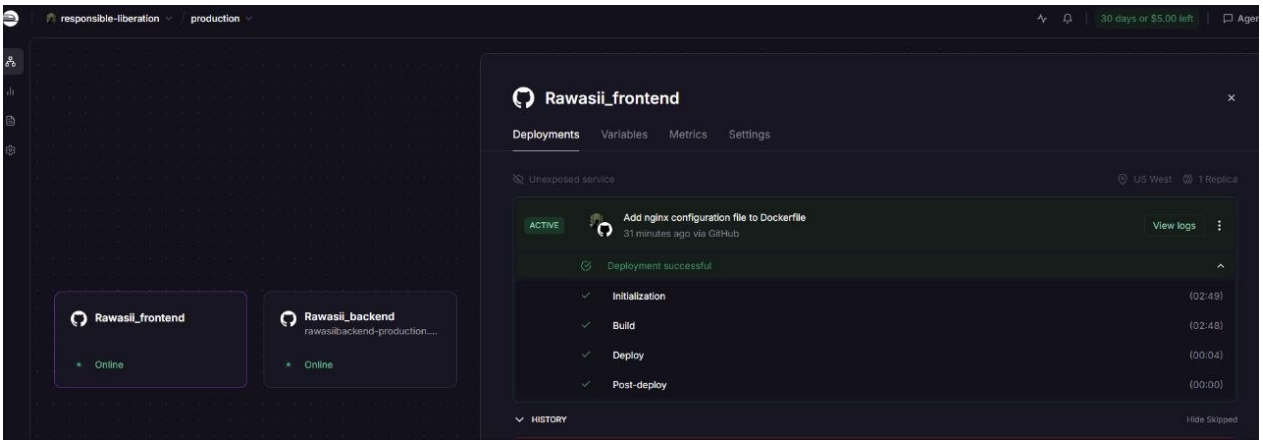
7.7 Deployment and Hosting

To deploy and host the backend system, Railway was used as the cloud hosting platform. Railway provides a simple deployment environment and offers free resources for testing purposes, including temporary storage and runtime usage that were sufficient for hosting and testing the project during development.

In order to deploy the application successfully, the backend source code was uploaded to GitHub, allowing Railway to access and deploy the project directly from the repository.

The deployment process also required the use of Docker. Docker was used to containerize the application and define the execution environment needed for the backend to run correctly on the hosting platform. Additional configuration files were created to manage dependencies, runtime settings, and deployment instructions required by Railway.

This deployment setup helped ensure that the backend could run consistently and be accessible online while simplifying the hosting and environment configuration process.



8 Conclusion

Our project centered around the design and development of a social platform dedicated to the promotion of Algerian architectural heritage, addressing a concrete problem: the absence of a unifying digital space allowing heritage enthusiasts to share knowledge, interact, and mobilize around a common cause.

We began with an in-depth analysis of the project specifications and existing similar applications, in order to identify the essential features to be integrated and the gaps to be filled. This phase was followed by a detailed design of the system architecture, accompanied by rigorous task planning and proactive risk anticipation.

Development was organized around a decoupled architecture: a RESTful backend built with Dart Frog and Supabase, handling user management, publications, social interactions and notifications, and a mobile frontend developed with Flutter. The application was containerized using Docker to facilitate deployment and system portability.

Throughout the project, continuous testing was carried out to ensure the reliability of the implemented features, and comprehensive technical documentation was produced to support both users and developers in working with the platform.

8.1 Skills Acquired

- Backend architecture: We gained practical mastery of RESTful API development, relational database management with Supabase, and the implementation of authentication and data security mechanisms.
- Mobile development: Building the frontend with Flutter allowed us to understand the specific challenges of developing modern, responsive, and user-friendly mobile interfaces.
- Project management: Coordinating between team members, meeting deadlines, and adapting to unexpected constraints strengthened our collaborative project management skills.
- DevOps and deployment: Setting up a Docker environment and exploring cloud deployment solutions introduced us to the real-world challenges of deploying a web application to production.
- Self-directed learning: Faced with technologies not covered in our curriculum, we developed the ability to learn independently from official documentation, thereby reinforcing our technical autonomy.

8.2 Perspectives

On the functional side, several improvement areas were identified but could not be integrated within the current scope of the project:

- Production deployment: The stable deployment of the application on a cloud infrastructure remains a short-term priority, requiring adaptation of the build environment to the Dart Frog framework.

- Personalized news feed: Implementing a recommendation algorithm based on each user's preferences and subscriptions would considerably enrich the browsing experience.
- Advanced moderation features: Developing more complete administrative tools, including detailed statistics and an automated reporting system, would strengthen community governance.

In the long term, the platform could evolve into a fully integrated digital ecosystem dedicated to Algerian heritage, incorporating advanced geolocation features, interactive mapping, and collaboration between cultural institutions, thereby contributing to the preservation and promotion of the national architectural heritage.